

Laboratory

Controlled Load Testing and Resilience Analysis of a Web API

Introduction

In real software projects, it is not enough for an API to work correctly from a functional perspective. It is also important to understand how the service behaves when the number of requests increases over time. A system may respond correctly under normal conditions, but its performance can degrade when it receives a high volume of traffic.

In this laboratory, each project group will work with the provided .NET API repository and will create a **console application** capable of sending repeated HTTP requests to the API through a **parameterized URL**. The purpose is to observe how the API behaves under increasing load in a **controlled local environment**, analyze the impact on performance, and prepare a technical report with graphs and conclusions.

This activity must be performed only on the students' **local machine** and only against the provided academic project or another local environment owned by the group. The purpose is academic analysis of **performance, scalability, and resilience**, not disruption of external systems.

Objective

The objective of this laboratory is to analyze the behavior of a .NET Web API under controlled local load conditions by generating repeated HTTP requests from a console application and evaluating how different levels of traffic affect the performance of the service.

At the end of the laboratory, students should be able to:

- Run the provided .NET API locally.
- Build a console application that sends HTTP requests to a parameterized API endpoint.
- Execute several controlled test rounds with different traffic conditions.
- Observe how the API performance changes as the load increases.
- Analyze the effect of the test on local machine resources such as CPU and memory.
- Present graphs and a technical report with conclusions about the API's degradation point and recommendations for improving resilience.

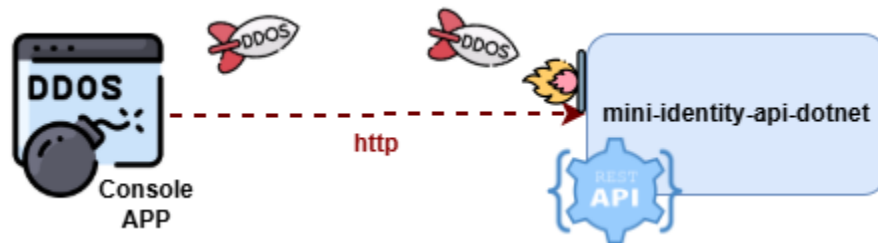
Repository to be used

The API to be tested is the following repository:

<https://github.com/wolfcor10/mini-identity-api-dotnet>

Each group must clone this repository, run it locally, and identify which endpoint will be used as the service under test.

General description of the activity



Each project group must create a **console application** that sends repeated HTTP requests to the API running locally. The URL of the endpoint must be configurable, so the application can target different routes if needed.

The group must then execute multiple controlled test rounds, varying relevant parameters such as the number of concurrent requests, frequency of requests, duration of each round, or total workload. During these test rounds, students must observe the impact on:

- response time
- number of successful and failed requests
- stability of the API
- CPU usage
- memory usage
- overall behavior of the local machine

The final purpose is to determine the point at which the API begins to show **significant degradation**, such as increased latency, errors, instability, or unresponsiveness.

Scope and safety conditions

This laboratory must follow these rules:

- The API must be tested **only on the local machine** or in an isolated private environment controlled by the group.
- Students must **not** direct the console application against public services, university systems, cloud services, or third-party infrastructure.
- The activity must be understood as a **controlled load and resilience experiment**, not as an attack against external services.
- The experiment must be stopped if the local machine becomes unstable or if the work environment is affected significantly.

What students must develop

Each project group must develop a **console application** that:

- accepts a configurable API URL
- sends repeated HTTP requests to the selected endpoint
- allows controlled variation of the test conditions
- records the number of requests performed
- records successful and failed responses
- records timing or performance-related observations
- allows the group to execute different rounds of testing and compare results

The application does not need to be highly complex, but it must be sufficient to support an organized experiment and generate measurable results.

Experimental work to be performed

Each group must perform several test rounds against the local API. The rounds must be organized progressively, from lower load to higher load, so that the behavior of the API can be compared under different levels of pressure.

For each round, the group must document:

- endpoint used
- machine where the test was executed
- relevant hardware information
- test duration
- request pattern used
- observed behavior of the API
- observed behavior of the local machine

- number of successful and failed requests
- relevant performance metrics collected

The objective is not simply to run requests, but to **analyze** how the service responds under growing demand.

Performance analysis to be included

Students must analyze the performance of the API in relation to the computational capacity of the machine used for the experiment.

The report should help answer questions such as:

- How does the response time change as the request volume increases?
- At what point do errors begin to appear?
- At what point does the system become unstable or show serious degradation?
- How do CPU and memory usage behave during the test?
- What is the maximum sustainable workload observed before the service becomes critically affected?

Students should identify the **degradation point** of the application and explain how they determined it.

Required graphs

The report must include clear graphs that support the analysis. At minimum, each group should present graphs related to:

- request load vs response time
- request load vs success rate
- request load vs failure count
- test round vs CPU usage
- test round vs memory usage

Groups may include additional graphs if they believe those graphs better support their analysis.

Expected report contents

Each project group must prepare a **PDF report** and submit it through the platform. The report must include:

1. Cover page

- course name
- laboratory title
- group members
- date

2. Introduction

A brief explanation of the purpose of the experiment and why performance analysis is important in APIs.

3. Objective

A clear statement of what the group intended to evaluate.

4. Test environment

Description of:

- local machine used
- operating system
- processor
- RAM
- relevant software tools

5. API under test

- repository used
- endpoint selected
- short explanation of why that endpoint was chosen

6. Console application description

A brief explanation of what the application does and how it was used for the experiment.

7. Test rounds

A description of each round, including the main parameters used.

8. Results

Tables and graphs showing the measured behavior.

9. Analysis

Interpretation of the results, including:

- stable behavior
- degradation signs
- critical behavior
- observed limitations of the machine or API

10. Conclusions

Final technical conclusions of the group.

11. Recommendations

Suggestions to improve the resilience or scalability of the API.

12. Repository link

The report must include the **Git repository link** of the console application developed by the group.

Deliverable

Each project group must submit the following through the platform:

- **one PDF report per group**
- **the Git repository link of the console application**

Required format

- The report must be submitted in **PDF**
- The work is submitted **by project group**
- The repository must be accessible for review

Expected deliverable summary

Each group must deliver:

- a local load-testing console application
- a PDF technical report
- graphs showing the observed performance behavior
- conclusions about the API degradation point
- the Git repository link of the console application

Evaluation focus

The laboratory will mainly evaluate:

- clarity of the experimental setup
- correctness of the console application purpose
- quality of the measurements and observations
- relevance and clarity of the graphs
- quality of the technical analysis
- clarity of conclusions and recommendations
- correct submission of the PDF report and Git repository link

Final note for students

This laboratory is intended to help students understand how an API behaves under increasing demand in a controlled and ethical academic context. The goal is to analyze performance and resilience, not to affect external systems. All tests must be performed locally and documented carefully.